

Continuous Fairness On Data Streams

ACM Reference Format:

. 2018. Continuous Fairness On Data Streams. In . ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

3 Continuous Fairness Monitoring

3.2 Fairness Checking

LEMMA 3.6. *Algorithm Monitor-BFair is optimal.*

PROOF. Let W be a window of size $|W| = k \cdot s$, and let the fairness query specify desired proportions $f(p)$ for each protected attribute $p \in \{1, \dots, \ell\}$. The goal of the query is to verify whether, in every block B_i of size s (for $1 \leq i \leq k$), the number of occurrences of each attribute p lies within the range $[\lfloor f(p) \cdot s \rfloor, \lceil f(p) \cdot s \rceil]$.

The sketch S stores cumulative counts of $\ell - 1$ protected attributes at each index $1 \leq i \leq |W|$. For any block B_i , the algorithm computes the frequency of each attribute as follows:

- For the first block B_1 , it uses the cumulative count stored at position s to compute the frequency of the first $\ell - 1$ attributes in B_1 .
- The frequency of the ℓ -th attribute is computed by subtracting the sum of the other $\ell - 1$ counts from the block size s .
- For each subsequent block B_i ($i > 1$), the algorithm computes the difference between the sketch values at positions $i \cdot s$ and $(i - 1) \cdot s$, which yields the frequency of the first $\ell - 1$ attributes in B_i .
- Again, the frequency of the ℓ -th attribute is inferred as above.

In each block, the algorithm compares the frequency of every attribute value p to its allowed range. If a single attribute falls outside the allowed range in any block, the algorithm terminates early and correctly returns **No**. Otherwise, if all blocks satisfy the fairness constraints, it returns **Yes**. Since the sketch correctly captures cumulative counts and the algorithm faithfully reconstructs the per-block frequencies and verifies them against the fairness bounds, it always returns the correct answer. $\square$

4 Experimental Evaluations

4.1 Fairness Analyses & Qualitative Study

4.1.1 *Fairness Analyses.* We compare our proposed reordering algorithm BFair-ReOrder with BruteForce to empirically evaluate our theoretical claim. We observe in Figure 1 that BFair-ReOrder returns the same number of unique fair blocks as that of BruteForce corroborating its optimality.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

We vary window size $|W|$, block size s , and cardinality ℓ of different datasets and present the percentage of unique blocks that become fair after BFair-ReOrder is applied on the original stream. Figure 2 presents all the results. A general observation is, BFair-ReOrder significantly improves the fairness of the data stream. In general, with increasing window size $|W|$, as well as block size s , fairness substantially improves, which is intuitive, because a larger window or block size increases the likelihood of more number of blocks satisfying fairness for the same constraints. On the other hand, with increasing cardinality, the group fairness requirement becomes more complex, leading to less percentage of fair blocks.

In Figure 3, we vary $|X|$ and measure the percentage of unique fair blocks after BFair-ReOrder is applied. The results indicate that with increasing $|X|$, the percentage of unique fair blocks increases - this is intuitive, since with increasing landmark items, it becomes more likely to satisfy fairness constraints, thereby increasing the overall percentage.

4.1.2 *Qualitative Study.* We conduct a qualitative study with 10 graduate students to compare perceived recommendation quality under block-level vs. window-level fairness in two online content recommendation applications: (1) **Song playlists**, consisting of 15 tracks from three genres (pop, electronic, and country) displayed over three pages (blocks), and (2) **ML content**, comprising 15 YouTube videos across three types (tutorial, project, and explainer) displayed over three pages (blocks).

For each task, two recommendation orders—one ensuring per-page (block-level) and the other overall (window-level) representation—are presented in randomized order, with participants blinded to the model used. Participants answer three questions for each application: (1) What is your average satisfaction with the list generated by Model A? (2) What is your average satisfaction with the list generated by Model B? (3) Between Model A and Model B, which list do you prefer overall?

Table 1 summarizes the qualitative feedback from participants comparing block-level and window-level fairness models across the two applications. Overall, participants reported noticeably higher satisfaction with the recommendations produced by the block-level model (average satisfaction above 4.2) compared to the window-level model (average satisfaction around 3.0). A strong majority of users (approximately 89%) preferred the block-level ordering in both the song playlist and ML content scenarios. These results suggest that enforcing fairness at the block level yields recommendations that are perceived as both fairer and more satisfying to users, particularly in stream settings.

4.2 Running Time of Fairness Algorithms

We first present the effectiveness of Monitor-BFair and BFair-ReOrder varying pertinent parameters. In Section 4.2.4, we separately compare implemented baselines with our solutions, exhibiting that the baseline is 2 – 3 order of magnitude slower than ours.

4.2.1 *Throughput.* In these experiments, we measure the throughput (per second) of Monitor-BFair by varying $|W|$, s , and ℓ , considering both scenarios: one where re-ordering involving landmark

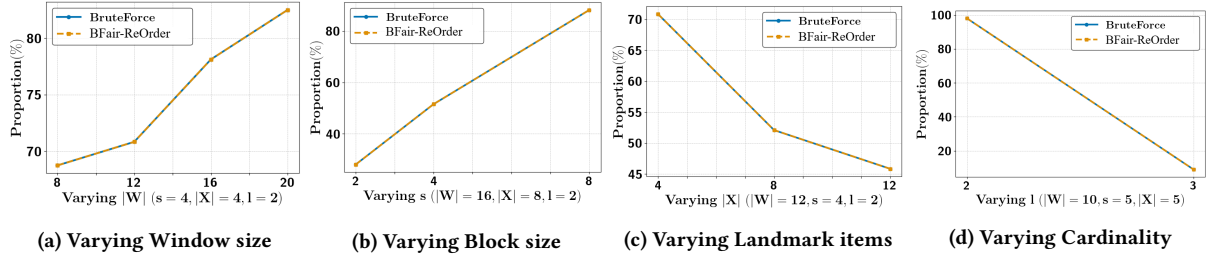


Figure 1: Qualitative Analysis: Performance against BruteForce

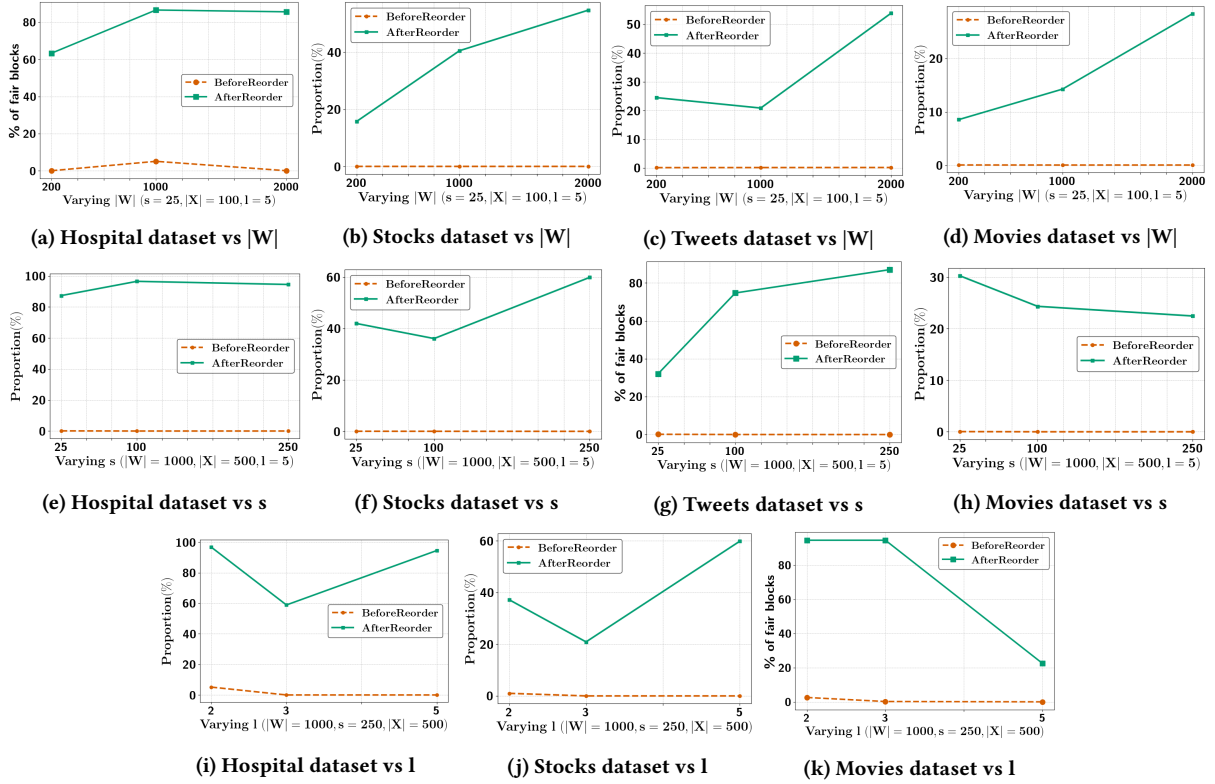


Figure 2: Fairness analysis

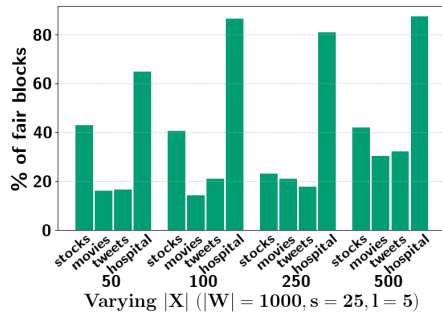


Figure 3: Effect of landmark on fairness

Application	Avg. Sat. (Block)	Avg. Sat. (Window)	% Prefer Blocks	% Prefer Windows
Song Playlist	4.22	3.00	89%	11%
ML Content	4.33	3.00	89%	11%

Table 1: User response comparison: block vs. window-level fairness.

items is required using BFair-ReOrder, and one where it is not. Figure 4a have these results and demonstrate that our algorithms exhibit very high throughput (as high as 40,000 at cases). As expected,

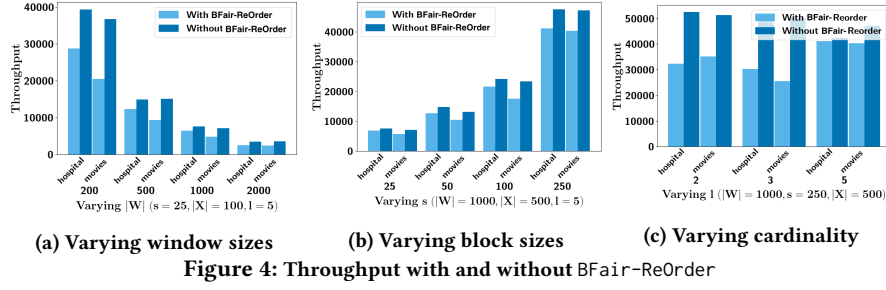


Figure 4: Throughput with and without BFair-ReOrder

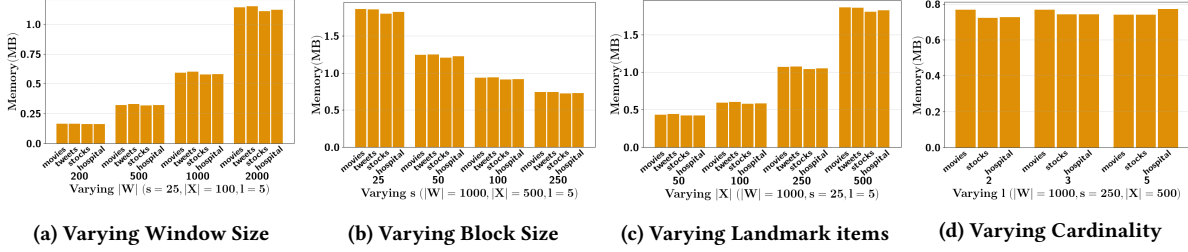


Figure 5: Query Processing with BFair-ReOrder Memory Consumption over 5000 windows

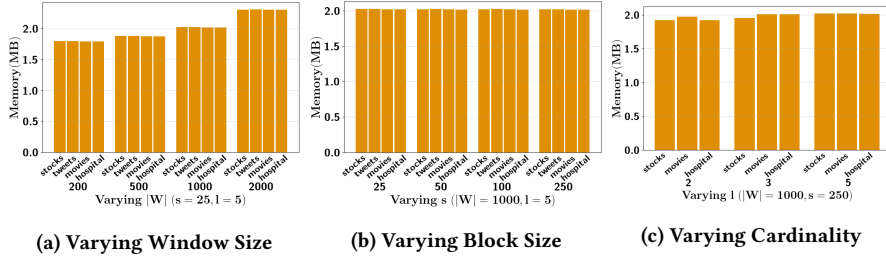


Figure 6: Query processing without BFair-ReOrder Memory Consumption over 5000 read-only windows

Monitor-BFair incurs higher processing time when reordering is required, resulting in lower throughput, and vice versa. We also observe an inverse relationship between $|W|$ and throughput. This is indeed true as the window size increases $|W|$, this leads to processing higher number of blocks, leading to more time. On the other hand, in Figure 4b, we observe the directly proportional relationship between s and throughput. Growing s decreases the number of blocks for a fixed $|W|$, thus decreasing the time taken for monitoring fairness and increasing the throughput. The impact of l is less pronounced (Figure 4c). While higher cardinality may increase running time, evenly distributed constraints often lead to more fair blocks, reducing reordering effort. Thus, overall running time stays stable with moderate l , and throughput remains nearly constant without BFair-ReOrder.

4.2.2 Tail Latency. We report the 90th percentile tail latency of Monitor-BFair over 5,000 windows, both with and without algorithm BFair-ReOrder, as shown in Figure 7.

As shown in Figure 7a, tail latency increases with larger $|W|$. For instance, the time to process a single fairness constraints at notes can exceed 10 ms when $W = 2000$. However, it is important to note

that this represents the slowest 10% of cases and does not reflect the typical running time.

Figure 7b illustrates the expected decrease in tail latency with increasing s , consistent with earlier observations.

Similarly, Figure 7c shows the effect of increasing l . Due to variations in the distribution of values across different $\mathcal{A}$, the tail latency does not exhibit a clear trend in this case.

Tail latency is significantly higher when BFair-ReOrder is needed inside Monitor-BFair, as reordering introduces substantial overhead compared to just monitoring. In contrast, the absence of reordering results in minimal delays, even for the slowest 10% of queries.

4.2.3 Running Time & Memory Consumption. Average running time is typically in the order of fraction of milliseconds, even when reordering is needed. Memory consumption of Monitor-BFair is negligible and consistent to the sketch memory consumption presented later.

4.2.4 Running Time compared to Baseline. We compare the runtime of BFair-ReOrder with that of BruteForce across varying values of $|W|$, s , and $|X|$, as shown in Figure 8. Due to the extreme

inefficiency of the BruteForce algorithm, the streaming environment automatically terminated the process (connection closed due to polling for over 5 minutes) when attempting experiments with window and landmark sizes above 24 items or cardinality beyond 3. Figure 8a shows the inefficiency of BruteForce: while comparable to BFair-ReOrder for $|W| \leq 16$, its runtime explodes beyond 10 s at $|W| = 20$, confirming its exponential complexity. BruteForce exhaustively searches all reorderings, making it highly sensitive to s (Figure 8b); larger s reduces blocks, increasing the chance of early fairness. Increasing $|X|$ similarly extends runtime but with a gentler slope. Overall, BFair-ReOrder is over three orders of magnitude faster. BSketchQP is consistently outperformed by Monitor-BFair, and those results are omitted for brevity.

4.3 Preprocessing Analyses

We present the preprocessing analyses of our solutions, and later in the section we present the comparison with appropriate baselines.

4.3.1 Sketch Construction & Update. We report the preprocessing time for 5,000 windows on the stocks and movies datasets in Figure 10, showing the time breakdown for sketch construction and updates, with and without BFair-ReOrder (please note if BFair-ReOrder is used the sketch needs to be constructed from scratch). Without BFair-ReOrder, sketch construction time is negligible, as it is built once and updated in the remaining windows. As shown in Figure 10a, preprocessing time grows with $|W|$ due to higher sketch construction cost, while update time remains stable. Varying s or ℓ has minimal impact (Figure 10b), showing robustness to block size and cardinality. At times, preprocessing can be slower without BFair-ReOrder, since batch memory access via landmark items (with fixed $|X| = 500$) and reduce overhead compared to loading new records per window.

4.3.2 Memory Requirement. Figure 12 shows average peak memory usage during preprocessing over 5,000 windows, reflecting the maximum memory used—including background processes—under varying parameters.

As expected, memory consumption of FSketch increases with $|W|$ (Figure 12a) due to larger sketches. It decreases with increasing s (Figure 12b), as larger blocks reduce the number of configurations needed during BFair-ReOrder, lowering memory usage. Similarly, memory usage increases with $|X|$, not due to sketch size, but because reordering longer streams in BFair-ReOrder incurs greater background memory overhead. These experiments corroborate our theoretical analyses, exhibiting that proposed FSketch is lean and bounded by window size.

4.3.3 Comparison with Baseline. We compare the preprocessing times of BSketch and FSketch in Figure 13 on the stocks dataset under varying parameters. FSketch consistently outperforms BSketch by an orders of magnitude, with the gap widening as window size increases (Figure 13a)—a result of Backward Sketch rebuilding the sketch from scratch for each window, unlike Forward Sketch, which supports efficient updates. As shown in Figures 13b and 13c, block size and cardinality have little impact, with minor variations attributed to platform-level fluctuations rather than the methods themselves.

4.4 Component-wise Running Time

We evaluate the average runtime of four core components — Sketch Construction, Sketch Update, BFair-ReOrder, and Monitor-BFair — on all the four dataset under varying $|W|$, s , and $|X|$. BFair-ReOrder (Figure 15) incurs the highest cost, followed by Sketch Construction, while Sketch Update and Monitor-BFair remain consistently lightweight. Sketch Construction scales linearly with $|W|$ but is largely insensitive to s and $|X|$. In contrast, BFair-ReOrder scales linearly with both $|W|$ and $|X|$, with runtime slightly decreasing as s increases.

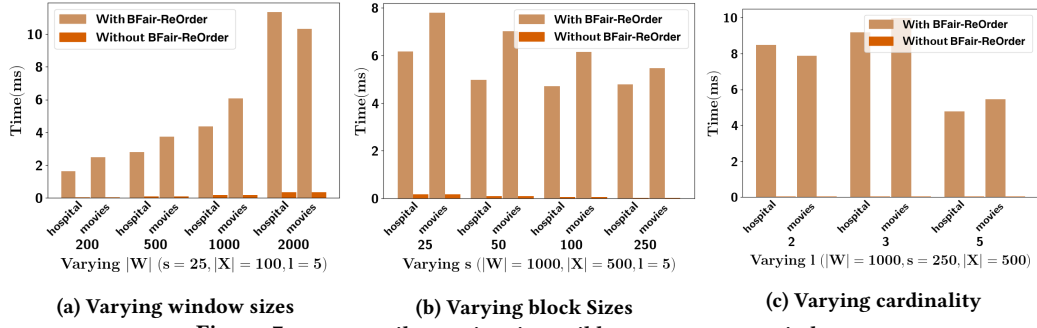


Figure 7: 90 percentile running time tail latency over 5000 windows

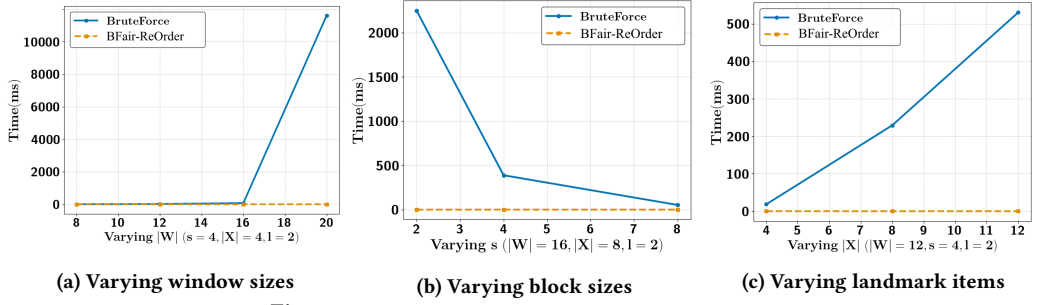


Figure 8: BruteForce vs BFair-ReOrder running time

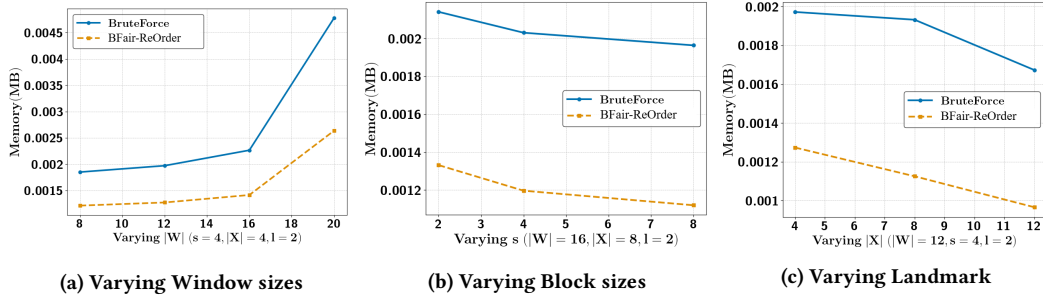


Figure 9: BruteForce vs p-FairReOrder Memory Consumption across different parameters

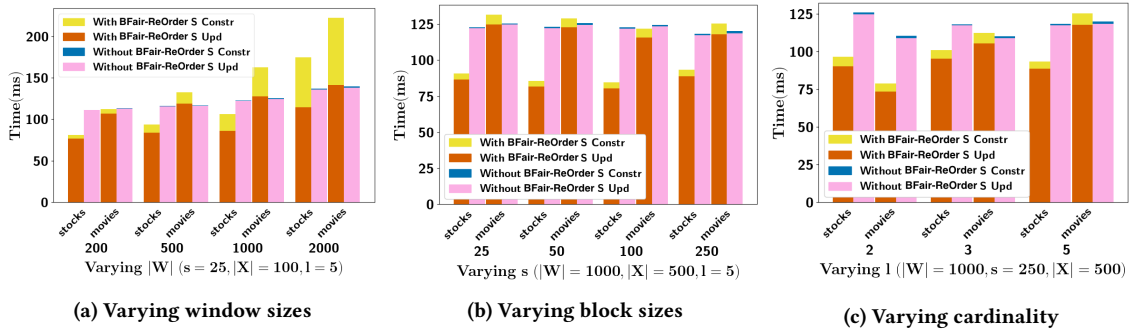


Figure 10: Total preprocessing running time for 5000 windows

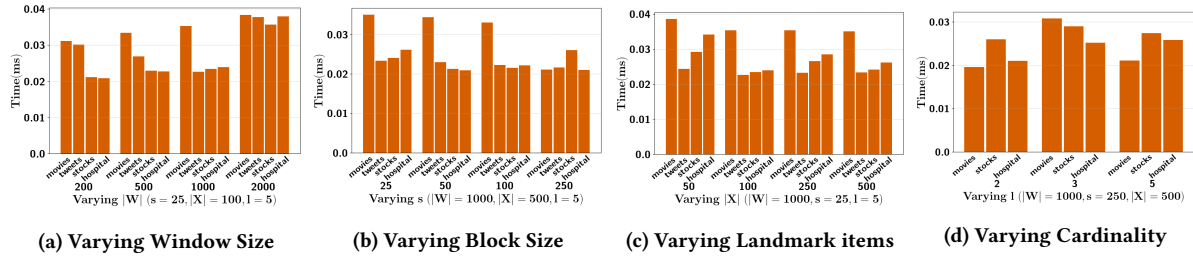


Figure 11: 90 percentile preprocessing tail latency over 5000 windows

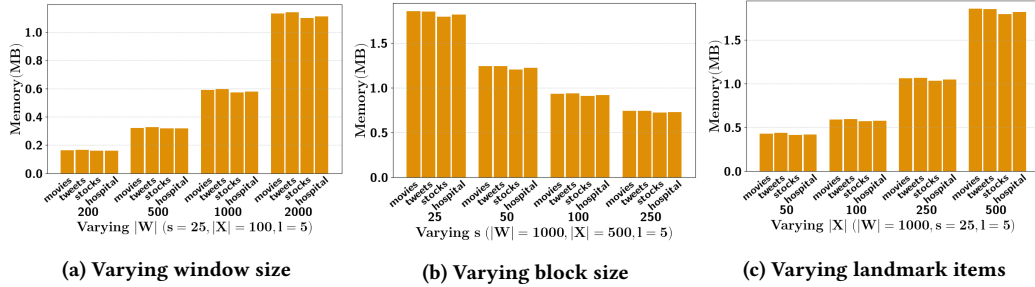


Figure 12: Average peak memory consumption over 5000 windows

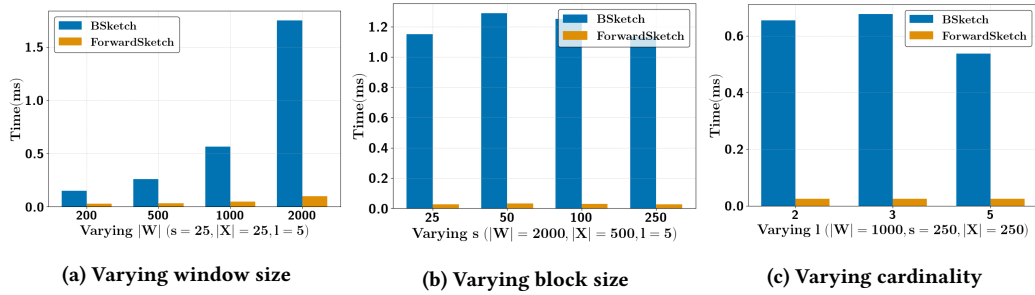
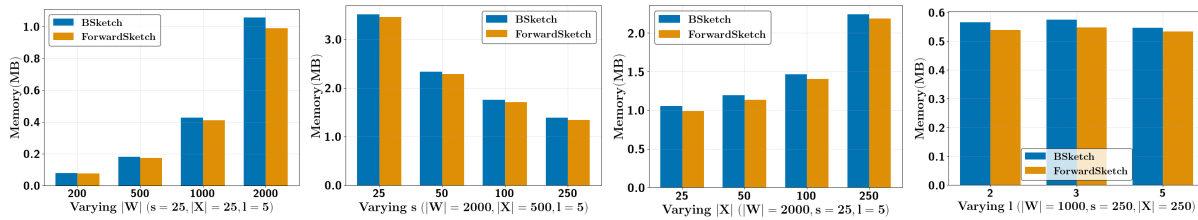


Figure 13: Preprocessing time of Backward Sketch vs Forward Sketch



(a) Average preprocessing peak consumption for varying window sizes (b) Average preprocessing peak consumption for varying block sizes (c) Average preprocessing peak consumption for varying landmark items (d) Average preprocessing peak consumption for varying cardinality

Figure 14: BSketch vs ForwardSketch Memory Consumption across different parameters

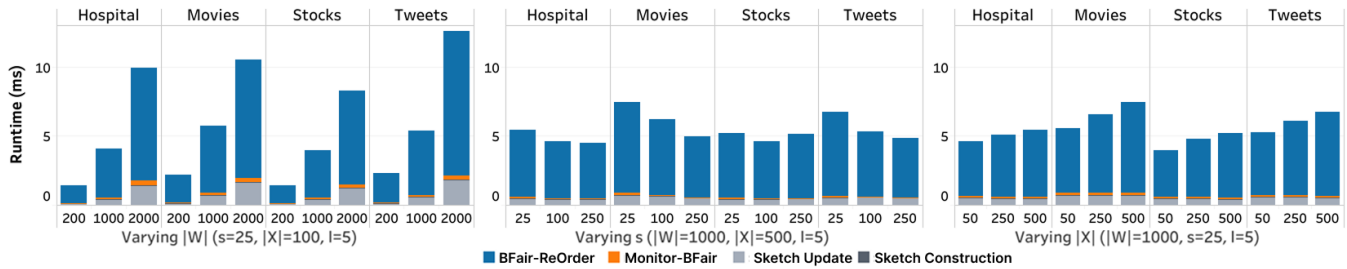


Figure 15: Component-wise Running Time